CSS @ A GLANCE: COMPREHENSIVE STUDY NOTES & CODE REFERENCE

-By Prof. Khushbu Patel

1: CSS Fundamentals, Architecture & Selectors

1.1 CSS Structure, Syntax & Rules

- **Structure or Syntax of CSS:** The syntax of CSS is slightly different from that of an HTML. CSS uses curly braces { }, colons (:) and semicolons (;).

```
selector{
    property : value;
    property : value;
    |          |
    property : value;
}
```

- **Importance of Semicolon:** A missing semicolon breaks cascading interpretation:

```
p{text-align:center background-color:tomato ;color:cyan}
```

- **• Only Last Valid Property Executes: Only 'color:cyan;' works.**
- **• Rendered Output: The <p> text becomes cyan color only; no background-color and align are applied.**

1.2 Types of CSS

Type of CSS	Description	Where It Is Written	Syntax (Example)
1. Inline CSS	Used to style a single HTML element directly. It has the highest priority.	Inside the HTML tag using the style attribute.	Heading
2. Internal (Embedded) CSS	Used to style elements within the same HTML page.	Inside the <style> tag in the <head> section of the HTML document.	<head><style> p { color: red; font-size: 18px; } </style></head>
3. External CSS	Used to apply styles to multiple web pages using a separate .css file.	In an external stylesheet linked using the <link> tag in the <head> section.	<head><link rel="stylesheet" href="style.css"></head> (style.css file contains CSS rules)

1.3 Various CSS Selectors

Selector Type	Description	Syntax	Example
Element Selector	Selects all elements of a specific type/tag. Lowest specificity (except universal).	element	p { color: black; }
ID Selector	Selects a single element with a specific id attribute. Very high specificity.	#id	#header { color: red; }

Class Selector	Selects elements with a specific class. Medium specificity.	.class	.menu { font-size: 16px; }
Attribute Selector	Selects elements based on an attribute or attribute value.	[attr], [attr=value]	input[type="text"] { border: 1px solid #ccc; }
Universal Selector	Selects all elements. Lowest specificity.	*	* { margin: 0; padding: 0; }
Descendant Selector	Selects elements that are descendants (any level) of a specified ancestor.	ancestor descendant	div p { color: blue; }
Child Selector	Selects elements that are direct children of a specified parent.	parent > child	ul > li { list-style: none; }
Grouping Selector	Groups multiple selectors and applies the same styles.	selector1, selector2	h1, h2, h3 { font-family: Arial; }

1.4 CSS Specificity (Priority Order)

- **Specificity Rule:** When multiple CSS rules target the same element, the browser decides which one to apply based on specificity (priority).

Priority Level	Selector Type	Example
1 (Highest)	Inline Styles	Heading
2	IDs	#title { color: red; }
3	Classes, pseudo-classes, attribute selectors	.title { color: green; }, :hover, [type='text']
4 (Lowest)	Elements and pseudo-elements	h1 { color: blue; }, ::before

CRITICAL SPECIFICITY RULES & EDGE CASES:
Key Specificity Conflict Scenarios & Exam Rules:
1. ID vs. Class: When the same property is applied through an ID and a class selector, the ID selector wins due to higher specificity. Example: .text { color: blue; } and #winner { color: red; } — Winner: ID (#winner), red.
2. Two IDs: An HTML element should not have two separate id attributes. Each element should have only one ID attribute. Example: Hello — Winner: Neither; invalid HTML usage.
3. Two Classes: An HTML element can have multiple classes. When two class selectors have equal specificity, the rule declared last wins. Example: .first { color: green; } and .second { color: orange; } — Winner: .second, orange.
4. ID vs. Multiple Classes: An ID selector has higher specificity than multiple class selectors. Example: .first { color: green; }, .second { color: orange; }, and #winner { color: red; } — Winner: #winner, red.

2: Typography, Box Model, Borders & Styling

2.1 Text Properties

- **Text Styling Purpose:** These properties are used to style, format, and control the appearance of text on a webpage, including alignment, spacing, transformation, decoration, font type, and size.

Property	Description	Syntax	Possible Values
color	Sets the color of the text	color: color;	Named colors (e.g., red), HEX (#ff0000), RGB (rgb(255,0,0)), RGBA
text-align	Aligns text inside an element	text-align: value;	left, right, center, justify, start, end
text-indent	Adds indentation to the first line	text-indent: length;	Any length (px, em, %)
text-transform	Controls text capitalization	text-transform: value;	none, capitalize, uppercase, lowercase
text-decoration	Adds decoration like underline or line-through	text-decoration: value;	none, underline, overline, line-through, underline overline
letter-spacing	Controls space between letters	letter-spacing: length;	Normal (normal), or custom spacing (px, em). Negative values are allowed.
word-spacing	Controls space between words	word-spacing: length;	Normal (normal), or custom spacing (px, em). Negative values are allowed.
line-height	Sets space between lines	line-height: value;	normal, number (e.g., 1.5), length (px, em), %. Negative values are not allowed.
text-shadow	Adds shadow to text	text-shadow: h-shadow v-shadow blur color;	none or values like 2px 2px 5px gray (can add multiple shadows separated by commas)

2.2 Font Properties & Google Fonts Embeds

Property	Description	Syntax	Example	Possible Values
font-family	Specifies the font type	font-family: "FontName", fallback;	font-family: "Arial", sans-serif;	Any font name: "Arial", "Times New Roman", "Courier New", generic families: serif, sans-serif, monospace, cursive, fantasy
font-size	Sets the font size	font-size: size;	font-size: 20px;	Length units: px, em, rem, %; Keywords: xx-small, x-small, small, medium, large, x-large, xx-large, smaller, larger
font-style	Defines style of text	font-style: value;	font-style: italic;	normal, italic
font-weight	Sets font thickness	font-weight: value;	font-weight: bold;	normal, bold, bolder, lighter, 100, 200, 300, 400, 500, 600, 700, 800, 900
font-variant	Displays text in small-caps	font-variant: value;	font-variant: small-caps;	normal, small-caps

2.3 CSS Borders & Image Styling

- **Border Purpose:** A border is a line that wraps around an HTML element's content and padding. It visually separates elements or highlights them on a web page.

Property	Description	Possible Values	Syntax / Example
border-style	Defines the style of the border (Must to Define).	none, solid, dashed, dotted, double Note: Multiple values allow different side-wise borders.	border-style: solid border-style: solid dashed double dotted
border-width	Sets the thickness of the border	length (px, em, rem)	border-width: 10px;
border-color	Sets the color of the border	Named colors (red), hex (#FF0000), RGB (rgb(255,0,0))	border-color: tomato;
border (shorthand)	Combines style, width, color in one line	border: width style color;	border: 2px solid red;
border-top	Sets border properties for the top side	width style color	border-top: 3px dashed blue;
border-right	Sets border properties for the right side	width style color	border-right: 2px solid green;
border-bottom	Sets border properties for the bottom side	width style color	border-bottom: 4px dotted orange;
border-left	Sets border properties for the left side	width style color	border-left: 5px double purple;

MANDATORY BORDER-STYLE RULE:
The default value of border-style is 'none'. Therefore, properties such as border-width and border-color will have NO visible effect unless border-style (e.g., solid, dashed, dotted) is explicitly defined first.
• Won't render: div {border-width: 4px; border-color: red;}
• Will render: div {border-style: solid; border-width: 4px; border-color: red; }

(Note: Unlike width and color, border-radius affects the element's actual shape and corners regardless of whether a visible border stroke is drawn.)

- **CSS Image Styling & Thumbnails:**

Property	Syntax	Example	What it does
border-radius	border-radius: radius;	border-radius: 20px;	Sets the corner rounding for all four corners.
border-top-left-radius	border-top-left-radius: radius;	border-top-left-radius: 15px;	Rounds only the top-left corner.
border-top-right-radius	border-top-right-radius: radius;	border-top-right-radius: 15px;	Rounds only the top-right corner.
border-bottom-right-radius	border-bottom-right-radius: radius;	border-bottom-right-radius: 15px;	Rounds only the bottom-right corner.
border-bottom-left-radius	border-bottom-left-radius: radius;	border-bottom-left-radius: 15px;	Rounds only the bottom-left corner.

padding	padding: value;	padding: 5px;	The border property and padding property are used to make a thumbnail image.
opacity	opacity: value;	opacity: 0.3;	To make an image transparent, we have to use the opacity property. The value lies between 0.0 (transparent) to 1.0 (Solid).
Responsive Image	max-width: %; height: auto;	max-width: 100%; height: auto;	The image becomes flexible — it scales down when screen is smaller, but won't stretch beyond natural size.

2.5 Box Properties & Box Sizing

Property	Description	Possible Values	Syntax / Examples
margin	Creates space outside the element (outside border). Used to separate elements on the page.	• auto -> browser calculates margin (useful for centering) • length -> px, em, %, negative values allowed	margin: 10px; Shorthand examples: • 4 values: margin: 25px 50px 75px 100px; (top=25, right=50, bottom=75, left=100) • 3 values: margin: 25px 50px 75px; (top=25, right/left=50, bottom=75) • 2 values: margin: 25px 50px; (top/bottom=25, right/left=50) • 1 value: margin: 25px; (all sides=25)
padding	Creates space inside the element (between content & border).	length -> px, em, %	padding: 10px; Shorthand examples: • 4 values: padding: 25px 50px 75px 100px; (top=25, right=50, bottom=75, left=100) • 3 values: padding: 25px 50px 75px; (top=25, right/left=50, bottom=75) • 2 values: padding: 25px 50px; (top/bottom=25, right/left=50) • 1 value: padding: 25px; (all sides=25)
width	Specifies width of an element's content area.	auto, length (px, em, %)	width: 50px;
height	Specifies height of an element's content area.	auto, length (px, em, %)	height: 50px;
box-sizing	Defines how width and height are calculated — whether padding and borders are included. Used to control layout behavior when adding padding/borders.	• content-box (default): width/height exclude padding & border. • border-box: width/height include padding & border.	box-sizing: content-box (height and width increase by increasing padding and border px add if padding, border-style, border-width mentioned) box-sizing: border-box (height and width will remain same)
box-shadow	Adds shadow effects around an element's frame. Used to add depth, hover effects, or visual highlights.	h-offset v-offset blur spread color	box-shadow: 2px 2px 5px gray;

3: Backgrounds

3.1 CSS Background Properties

Property	Description	Key Values / Notes	Syntax
background-color	Sets the background color of an element	Any valid color (red, #ffc0cb, rgb(255,0,0))	background-color: color;
background-image	Sets an image as the background	Use image URL;	background-image: url("image.jpg");
background-repeat	Controls how background image repeats	repeat, repeat-x, repeat-y, no-repeat (default repeats if not specified)	background-repeat: repeat;
background-position	Sets starting position of background image	top, bottom, left, right, center, x% y%	background-position: top;
background-size	Specifies the size of the background image	auto, cover, contain, width height	background-size: auto;
background-attachment	Sets whether background scrolls with content	scroll, fixed, local	background-attachment: scroll;
background-clip	Determines how far the background extends	border-box, padding-box, content-box	background-clip: border-box;
background (shorthand)	Combines color, image, repeat, position, size, attachment	Example: background: pink url("scenary.jfif") no-repeat top right cover fixed;	background: [color] [image] [repeat] [position] [size] [attachment];

4: Pseudo-Classes & Pseudo-Elements

4.1 Pseudo-Classes Reference

- **A. Pseudo-Classes:** Targets elements based on state, interaction, or DOM hierarchy.

Pseudo-class	What It Does	Syntax / Example
:hover	Changes color / styling when mouse hovers	selector:hover { prop: value; } p:hover { color: blue; }
:active	Changes color / styling when clicked	a:active { prop: value; }
:focus	Highlights input field when clicked / focused	input:focus { prop: value; }
:link / :visited	Colors for unvisited and visited links respectively	a:link { prop: value; } a:visited { prop: value; }
:nth-child(n)	Targets the nth element/child item	li:nth-child(2) { prop: value; }
:not(.special)	Styles all elements except those matching selector	p:not(.classname) { prop: value; }

4.2 Pseudo-Elements Reference

- **B. Pseudo-Elements:** Styles specific sub-parts of an element or injects generated cosmetic content.

Pseudo-element	Description	Syntax	Example	Result / Effect
::first-line	Styles first line of block-level element.	p::first-line { property: value; }	p::first-line { color: red; font-weight: bold; }	First line of the paragraph appears red and bold.
::first-letter	Styles first letter of block-level element.	p::first-letter { property: value; }	p::first-letter { font-size: 50px; color: red; }	First letter of the paragraph becomes large and red.
::before	Inserts content before an element's actual content.	selector::before { content: "..."; }	p::before { content: "Note: "; color: red; }	Adds "Note: " before each paragraph.
::after	Inserts content after an element's actual content.	selector::after { content: "..."; }	p::after { content: " ✓"; color: green; }	Adds a green checkmark after the paragraph.
::marker	Styles marker (bullet/number) of list items.	li::marker { property: value; }	li::marker { color: red; font-size: 20px; }	List bullets or numbers appear red and large.
::selection	Styles highlighted text selected by user.	::selection { property: value; }	::selection { background: blueviolet; color: white; }	Selected text appears white on blue-violet background.

Example 1: CSS Pseudo-Classes and Selectors

Task Specification: Design a webpage using HTML and internal CSS that includes a button, a text input field, and an unordered list. Apply the following styles:

• **Requirement 1:** Change the button's background and text colors on hover, and change its background color when clicked.

• **Requirement 2:** Change the text input's background color and border when it receives focus.

• **Requirement 3:** Style the third list item with blue text and an underline using :nth-child().

• **Requirement 4:** Apply a yellow background and bold font to the list item with the class highlight.

• **Requirement 5:** Apply a light gray background and italic font to all other list items using :not().

```
<!DOCTYPE html>
<html>
<head>
    <title>CSS Pseudo-Classes</title>
    <style>
        /* Button styles */
        button {
            background-color: lightblue;
            color: black;
            padding: 10px 20px;
            border: none;
            cursor: pointer;
        }
        button:hover {
            background-color: navy;
            color: white;
        }
        button:active {
            background-color: orange;
        }
        /* Input focus */
        input:focus {
            background-color: lightgreen;
```

```
            border: 2px solid green;
            outline: none;
        }
        /* Third list item */
        ul li:nth-child(3) {
            color: blue;
            text-decoration: underline;
        }
        /* Highlighted item */
        ul li.highlight {
            background-color: yellow;
            font-weight: bold;
        }
        /* All items except highlight */
        ul li:not(.highlight) {
            background-color: lightgray;
            font-style: italic;
        }
    </style>
</head>
<body>
    <h2>CSS Pseudo-Classes</h2>
    <button>Hover or Click Me</button>
    <br><br>
    <label for="name">Name:</label>
    <input type="text" id="name" placeholder="Click to focus">
    <h3>My List</h3>
    <ul>
        <li>First item</li>
        <li>Second item</li>
        <li>Third item</li>
        <li class="highlight">Highlighted item</li>
        <li>Fifth item</li>
    </ul>
</body>
</html>
```

Example 2: CSS Pseudo-Elements

Task Specification: Design a webpage using HTML and internal CSS that contains headings, paragraphs, and an unordered list. Apply the following styles:

- **Requirement 1:** Use ::first-line to display the first line of a paragraph in bold and dark blue.
- **Requirement 2:** Use ::first-letter to display the first letter of a paragraph in a larger red font.
- **Requirement 3:** Use ::before to insert a star symbol before a heading.
- **Requirement 4:** Use ::after to insert a check mark after a heading.
- **Requirement 5:** Use ::marker to change the color and size of the unordered-list bullets.
- **Requirement 6:** Use ::selection to display selected text with a purple background and white text.

```
<!DOCTYPE html>
<html>
<head>
    <title>CSS Pseudo-Elements</title>
    <style>
        /* First line of paragraph */
        p::first-line {
            color: darkblue;
            font-weight: bold;
        }
        /* First letter of paragraph */
```

```
        .intro::first-letter {
            color: red;
            font-size: 32px;
            font-weight: bold;
        }
        /* Before heading */
        h2::before {
            content: "★ ";
            color: orange;
        }
        /* After heading */
        h2::after {
            content: " ✓";
            color: green;
        }
        /* List bullet styling */
        ul li::marker {
            color: red;
            font-size: 20px;
        }
        /* Selected text styling */
        ::selection {
            background-color: purple;
            color: white;
        }
    </style>
</head>
<body>
    <h2>About CSS</h2>
    <p class="intro">
        CSS is used to style HTML webpages.
        It controls colors, fonts, spacing, and layout.
    </p>
    <p>
        Pseudo-elements allow us to style specific parts
        of HTML elements.
    </p>
    <h2>My Favorite Subjects</h2>
    <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
    </ul>
</body>
</html>
```

5: Modern Layout Systems: Display, Flexbox, Grid & Positioning

5.1 Display Property

Display Type	Description	Behavior / Use Case	Visual Behavior	Syntax
inline	Displays elements in a line, without starting on a new line.	Does not accept width/height. Common for , <a>, .	Elements sit side by side in a single line.	display: inline;
block	Displays element as a block, starting on a new line.	Takes full width available and allows width/height to be set.	Each element appears on a new line.	display: block;
inline-block	Combines features of inline and block.	Appears inline, but allows width and height.	Boxes are side by side, but size-controllable.	display: inline-block;
none	Hides the element completely (removed from layout).	Element takes no space on the page.	The element is invisible and does not occupy space.	display: none;

5.2 Flexbox Layout

Property	Description	Possible Values	Syntax / Example
display: flex	Defines a flex container to arrange items flexibly.	flex, inline-flex	div { display: flex; }
flex-direction	Defines the direction of flex items.	row (default), row-reverse, column, column-reverse	flex-direction: row;
justify-content	Aligns items horizontally (along the main axis).	flex-start, flex-end, center, space-between, space-around, space-evenly	justify-content: space-between;
align-items	Aligns items vertically (along the cross axis).	stretch (default), flex-start, flex-end, center	align-items: center;
flex-wrap	Determines whether flex items wrap onto multiple lines.	nowrap (default), wrap, wrap-reverse	flex-wrap: wrap;
gap	Defines the space between flex items.	Any CSS length unit (px, em, %)	gap: 15px;
order	Specifies the display order of flex items.	Integer values (0 default, can be positive or negative)	order: 2;
flex (shorthand)	Sets how an item grows, shrinks, and defines its base size.	flex: grow shrink basis; e.g., flex: 1 0 100px;	flex: 1; or flex: 1 0 200px;

5.3 Grid Layout Properties

Property	Description	Possible Values	Default Value	Syntax / Example
grid-template-columns	Defines the number and width of columns in a grid layout.	Fixed units (px, em, %)	none	grid-template-columns: 200px 1fr 2fr;
grid-template-rows	Defines the number and height of rows in a grid layout.	Fixed units (px, em, %)	none	grid-template-rows: 100px auto 100px;
gap (grid-gap)	Defines the space between rows and columns.	Any CSS length (px, em, %)	0	gap: 10px; or gap: 20px 40px;
grid-column	Specifies the horizontal position of an item within the grid columns (start / end lines).	<start-line> / <end-line>	auto	grid-column: 1 / 3; (spans across 2 columns)
grid-row	Specifies the vertical position of an item within the grid rows (start / end lines).	<start-line> / <end-line>	auto	grid-row: 1 / 2; (spans across 1 row)

5.4 CSS Positioning Properties & Stacking Context

Property	Description	Possible Values	Default Value	Syntax / Example
position	Defines how an element is positioned in the document.	static, relative, absolute, fixed, sticky	static	position: absolute;
top	Distance between element and top edge of container.	Any CSS length (px, %, auto)	auto	top: 20px;
right	Distance between element and right edge of container.	Any CSS length (px, %, auto)	auto	right: 10px;
bottom	Distance between element and bottom edge of container.	Any CSS length (px, %, auto)	auto	bottom: 15px;
left	Distance between element and left edge of container.	Any CSS length (px, %, auto)	auto	left: 30px;
z-index	Controls stack order of overlapping elements.	Integer (auto, positive or negative values)	auto	z-index: 2; (Higher = on top)

- **Operational Position Behaviors:**
- • **static:** Default. Element follows the normal document flow.
- • **relative:** Moved relative to its normal position (shifts top/bottom/left/right from its aligned position).
- • **absolute: Positioned relative to the nearest positioned ancestor (non-static container).**
- • **fixed: Stays fixed in place relative to the browser viewport even when scrolling.**
- • **z-index:** Controls which element appears on top of others in overlapping contexts.

6: CSS Multiple Column Layout

Property	Syntax	Example	What it does
column-count	column-count: number;	column-count: 4;	Specifies the number of columns an element should be divided into.
column-gap	column-gap: number;	column-gap: 20px;	Specifies the gap between the columns.
column-rule-style	column-rule-style: value;	column-rule-style: dashed;	Specifies the style of the rule between columns (none, dotted, dashed, solid, double; default: none).
column-rule-color	column-rule-color: color;	column-rule-color: rgb(219, 12, 47);	Specifies the color of the rule between columns.
column-rule-width	column-rule-width: value;	column-rule-width: 7px;	Specifies the width of the rule between columns.
Shorthand column-rule	column-rule: width style color;	column-rule: 1px solid cyan;	Specifies width, style, and color of the column dividing rule.

7: Transforms, Transitions, Animations & Gradients

7.1 2D Transforms & Transformations

Transform Function	Syntax	Example	What it does

translate(x, y)	transform: translate(x, y);
	or
	transform: translate(50px);
		transform: translate(50px, 30px);	Moves an element 50px right and 30px down.
			ormoves an element 50 px on x-axis only

translateX(n)	transform: translateX(n);	transform: translateX(100px);	Moves an element 100px horizontally (right).
translateY(n)	transform: translateY(n);	transform: translateY(-50px);	Moves an element 50px upward.
rotate(angle)	transform: rotate(angle);	transform: rotate(45deg);	Rotates the element 45° clockwise.
rotateX(angle)	transform: rotateX(angle);	transform: rotateX(45deg);	Rotates the element 45° in x axis only
rotateY(angle)	transform: rotateY(angle);	transform: rotateY(45deg);	Rotates the element 45° in Y axis only.
scale(x, y)	transform: scale(x, y);	transform: scale(1.5, 0.8);	Increases width 1.5× and decreases height to 0.8×.
scaleX(n)	transform: scaleX(n);	transform: scaleX(2);	Doubles the element's width.
scaleY(n)	transform: scaleY(n);	transform: scaleY(0.5);	Reduces the element's height by half.
skew(x, y)	transform: skew(x, y);	transform: skew(20deg, 10deg);	Slants an element 20° horizontally and 10° vertically.

skewX(angle)	transform: skewX(angle);	transform: skewX(20deg);	Skews an element horizontally by 20°.
skewY(angle)	transform: skewY(angle);	transform: skewY(20deg);	Skews an element vertically by 20°.

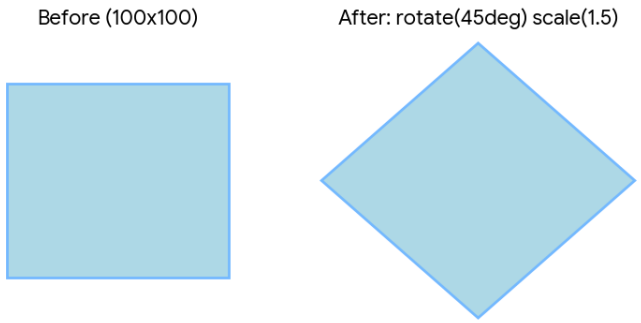
- **Transform Mirroring & Multiple Actions Notes:**
  - **Flip Horizontally: The transform: scaleX(-1) property is used to flip the image horizontally.**
  - **Mirror Vertically: The transform: scaleY(-1) property creates a mirror image vertically.**
  - **Full Mirror: The transform: scale(-1) property creates a mirror image vertically as well as horizontally.**

Combining Multiple Actions: Apply multiple transforms in a single statement: **Multiple values handle by space**

```
transform: rotate(30deg) scale(1.5) translate(100px, 100px) skew(20deg, 20deg);
```

Example 3: On Hover Effect (div:hover): On Hover Effect (div:hover) of the Div of a 100px by 100px blue box the box rotates 45 degrees, scales up by 1.5 times, and shifts 50px from x and y axis.

```
<head>
<style>
div {
  margin: 100px;
  width: 100px;
  height: 100px;
  background-color: lightblue;
}
div:hover {
  transform: rotate(45deg) scale(1.5) translate(50px, 50px);
}
</style>
</head>
<body>
<div></div>
</body>
```



Visual Transformation Result (Before & After Hover)

7.2 CSS Transitions & Shorthand Breakdown

Property	Syntax	Example	What it does
transition-property	transition-property: all | property | property1, ...;	transition-property: background-color, transform;	Specifies which CSS properties will be animated during transition.
transition-duration	transition-duration: time; (Must mention)	transition-duration: 0.5s;	Specifies how long the transition takes to complete.
transition-delay	transition-delay: time;	transition-delay: 0.3s;	Specifies how long to wait before starting the transition.
transition-timing-function	linear | ease | ease-in | ease-out | ease-in-out	transition-timing-function: ease-in-out;	Controls the speed curve of the transition over time.
transition (shorthand)	transition: property duration timing-function delay;	transition: background-color 0.5s ease-in-out 0.2s;	Sets all transition properties in a single line.

- **Transition Timing Functions Summary:**
  - **ease:** starts slow, speeds up, slows down (default)
  - **linear:** constant speed
  - **ease-in:** starts slow
  - **ease-out:** ends slow
  - **ease-in-out:** starts & ends slow
  - **steps(n):** jumps in n equal steps (Ref)
  - **cubic-bezier(x1, y1, x2, y2):** custom curve (Ref)

ESSENTIAL TRANSITION RULES:
Transition Rules: If transition-duration is not specified, transition has NO effect (default is 0s).
Multiple Transitions: Separate multiple transitions with commas: transition: width 2s 2s, height 2s 5s; (width starts after 2 s, height after 5s, both running over 2s).

Property	Required in Shorthand?	Default Value
transition-property	No	all
transition-duration	Yes	0s
transition-timing-function	No	ease
transition-delay	No	0s

Example 4: Create a CSS program that demonstrates the use of multiple transitions on a single element. When the user hovers over the div, it should:

- Increase in size,
- Change background color,
- Rotate 360 degrees,
- Become circular, all with smooth timing, delay, and different transition durations.

```
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background: rgb(142, 7, 233);

  /* Transition for multiple properties including transform */
  transition-property: width, height, transform, background-color,border-radius;
  transition-duration: 3s, 5s, 4s,6s,6s;
  transition-delay: 2s; /* Wait 2s before starting transition */
  transition-timing-function: ease-in-out; /* Smooth start and end */
}
div:hover {
  width: 150px;
  height: 150px;
  transform: rotate(360deg);
  background-color: yellow;
  border-radius:50%;
}
</style>
</head>
<body>
  <div></div>
</body>
```

7.3 CSS Animation & Keyframes

- **The @keyframes Rule:** Gradually changes styles from current to new style at specified keyframe intervals. Animation must be bound to an element.
-
- **Keyframe Percentages: from = 0% (start) and to = 100% (end); percentage points define intermediate animation stages.**

```
@keyframes animation-name {
    from { background-color: red; }
    to { background-color: yellow; }
}
/* Or using percentages */
@keyframes moveBox {
    0%   { top: 0px; left: 0px; }
    50%  { top: 100px; left: 200px; }
    100% { top: 0px; left: 0px; }
}
```

Property	Syntax	Example	What it does
animation-name	animation-name: keyframename;	animation-name: moveBox;	Specifies the name of the @keyframes to link to the element.
animation-duration	animation-duration: time;	animation-duration: 2s;	Specifies the time it takes for one animation cycle to complete.
animation-delay	animation-delay: time;	animation-delay: 1s;	Specifies the delay before the animation starts.
animation-timing-function	linear | ease | ease-in | ease-out | ease-in-out	animation-timing-function: ease-in-out;	Controls the speed curve of the animation over time.
animation-iteration-count	number | infinite	animation-iteration-count: infinite;	Specifies how many times the animation should repeat.
animation-direction	normal | reverse | alternate | alternate-reverse	animation-direction: alternate;	Specifies whether animation runs forward, backward, or alternates.
animation (shorthand)	animation: name duration timing-function delay iteration-count direction;	animation: example 3s ease-in-out 1s infinite alternate;	Sets all animation properties in a single line (shorthand form).

Example 5 **Animation for Position and Color Change:** The following example will change both the background-color and the position of the element when the animation is 25% completed, 50% completed, 75% completed and again when the animation is 100% completed:

```
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background-color: pink;
  position: relative;
  animation-name: example;
  animation-duration: 10s;
  animation-delay: 2s;
  animation-iteration-count: infinite;
}
@keyframes example {
  0%   {background-color:red; left:0px; top:0px;}
  25%  {background-color:yellow; left:200px; top:0px;}
  50%  {background-color:blue; left:200px; top:200px;}
  75%  {background-color:green; left:0px; top:200px;}
  100% {background-color:red; left:0px; top:0px;}
}
</style>
</head>
<body>
<div></div>
</body>
</html>
```

7.4 CSS Gradients (Linear, Radial & Conic)

- **A. Linear Gradients:** Linear gradients transition smoothly between colors along a linear direction.

Type	Syntax	Example	What it does
Linear Gradient (Default: Top to Bottom)	background-image: linear-gradient(color1, color2);	background-image: linear-gradient(purple, yellow);	Creates a smooth color transition from top to bottom (default direction).
Bottom to Top	background-image: linear-gradient(to top, color1, color2);	background-image: linear-gradient(to top, purple, yellow);	Creates a gradient starting from bottom to top.
Left to Right	background-image: linear-gradient(to right, color1, color2);	background-image: linear-gradient(to right, purple, yellow);	Creates a gradient moving from left to right.
Right to Left	background-image: linear-gradient(to left, color1, color2);	background-image: linear-gradient(to left, purple, yellow);	Creates a gradient moving from right to left.
Diagonal (to top left)	background-image: linear-gradient(to top left, color1, color2);	background-image: linear-gradient(to top left, purple, yellow);	Creates a diagonal gradient moving from bottom right to top left.
Diagonal (to bottom right)	background-image: linear-gradient(to bottom right, color1, color2);	background-image: linear-gradient(to bottom right, purple, yellow);	Creates a diagonal gradient from top left to bottom right.
Angle-based Gradient	background-image: linear-gradient(angle, color1, color2);	background-image: linear-gradient(90deg, pink, lightblue);	Uses degrees (0°, 90°, 180°, -90° etc.) to control gradient direction.
Evenly spaced multiple colors	linear-gradient(color1, color2, color3, ...)	linear-gradient(red, yellow, green);	Colors are evenly spaced between start and end points.
Non-evenly spaced colors	linear-gradient(color1 %, color2 %, ...)	linear-gradient(pink 20%, lightgreen 30%, lightblue 50%);	Uses percentage values to control how far each color extends.
Rainbow Gradient Example	linear-gradient(to right, ...colors...)	linear-gradient(to right, red, orange, yellow, green, blue, indigo, violet);	Creates a rainbow-style horizontal gradient.

- **B. Radial Gradients:** Radial gradients radiate outward from a central point.

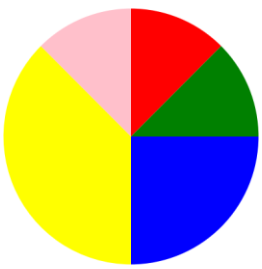
Type	Syntax	Example	What it does
Basic Radial Gradient	background-image: radial-gradient(color1, color2, color3);	background-image: radial-gradient(red, yellow, blue);	Creates a circular gradient radiating outward from the center.
Different Color Stops	background-image: radial-gradient(color1 %, color2 %, color3 %);	background-image: radial-gradient(yellow 10%, red 25%, pink 40%);	Defines uneven spacing for color transitions.
Ellipse (default)	background-image: radial-gradient(ellipse, color1, color2);	background-image: radial-gradient(purple, yellow, pink);	Creates an oval-shaped gradient that fills the container.
Circle Shape	background-image: radial-gradient(circle, color1, color2);	background-image: radial-gradient(circle, blue, yellow, pink);	Creates a perfectly circular gradient.

- **C. Conic Gradients:** Conic gradients rotate colors around a central anchor point (pie chart style).

Type	Syntax	Example	What it does
Basic Conic Gradient	background-image: conic-gradient(color1, color2, color3);	background-image: conic-gradient(red, yellow, blue);	Creates a circular gradient where colors rotate around the center point.
Using Degrees for Each Color	background-image: conic-gradient(color1 deg, color2 deg, ...);	background-image: conic-gradient(red 45deg, yellow 90deg, green 180deg);	Defines starting angles for each color transition.
Defined Start and End Degrees (Pie Chart Style)	background-image: conic-gradient(color1 start end, color2 start end, ...);	background-image: conic-gradient(red 0deg 90deg, yellow 90deg 180deg, blue 270deg);	Creates solid color segments similar to pie chart slices.
Rounded Pie Chart	border-radius: 50%; conic-gradient(...);	background-image: conic-gradient(red, yellow, green, blue); border-radius: 50%;	Turns conic gradient into a circular pie chart.
With 'from' Angle	background-image: conic-gradient(from angle, color1, color2, ...);	background-image: conic-gradient(from 90deg, red, yellow, green);	Rotates where the gradient starts based on the specified angle.

Example 6 **Conic Gradient with Specified 'from' Angle:** Example - conic gradient with specified from angle to create distinct colored segments rather than a smooth gradient transition.

```
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(from 90deg, red 0deg, red 45deg, green 45deg, green 90deg, blue
90deg, blue 180deg, yellow 180deg, yellow 315deg, pink 315deg);
  border-radius: 50%;
}
</style>
</head>
<body>
<div id="grad1"></div>
</body>
</html>
```



Conic Gradient Pie Chart Output

8: CSS Variables & Responsive Web Design

8.1 CSS Variables (Custom Properties)

- **Concept: CSS variables store reusable values that can be updated from one place. Accessed using the var() function, e.g. color: var(--b);. Case sensitive; local variables override global variables.**
- **Global Scope (in :root):** Defined inside :root, accessible across all elements of the document:

```
:root {    --b: blue;}
p {    color: var(--b);}
```

- **Local Scope (inside selectors):** Defined inside an element selector; scoped exclusively to that element:

```
p {    --b: blue;    color: var(--b);}
```

Category	Property / Syntax	Values / Example	Description / Usage
Definition	--variable-name: value;	--main-color: blue;	Defines a CSS variable (custom property). Must start with two dashes (--).
Usage	var(--variable-name)	color: var(--main-color);	Retrieves the value of the variable.
Global Variable	Declared inside :root	css :root { --text-color: green; }	Can be accessed by all elements throughout the document.
Local Variable	Declared inside a selector	css p { --border: solid; }	Accessible only within that selector's scope.
Case Sensitivity	Variable names are case-sensitive	--Color ≠ --color	Ensure consistent naming.
Overriding (Local > Global)	Local variable overrides global	css :root { --b: blue; } p { --b: brown; color: var(--b); }	Local value (brown) will apply to <p> instead of global (blue).

8.2 Responsive Viewport Meta Configuration

- **Viewport Meta Tag Requirement:** To make web pages responsive across phones, tablets, and desktops, include the viewport tag in <head>:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Attribute / Setting	Operational Meaning
name="viewport"	Tells the browser that this meta tag controls the viewport (visible area of the web page).
content="width=device-width"	Sets page width to match device screen width (prevents fixed desktop zoom-out).
initial-scale=1.0	Sets initial zoom level when page is first loaded (1.0 = 100% zoom).

8.3 CSS Media Queries Reference

Category	Property / Syntax	Example	Description / Usage
Basic Syntax	@media not|only mediatype and (mediafeature) and|or (mediafeature) { ... }	@media screen and (max-width: 600px) { ... }	Standard structure for defining responsive breakpoints and rules.
Media Types	all, screen	@media screen { ... }	Targets display output types.
Media Feature	(orientation: portrait) Or (orientation: landscape) Or (max-width: 500px) Or (min-width: 500px)	@media (orientation: portrait) { ... }	Adjusts styles based on screen width, height, or device orientation.
Combining Conditions	and, or, not	@media screen and (min-width: 600px) and (orientation: landscape) { ... }	Combines multiple media features for precise breakpoint control.
Range Example	@media (min-width: 500px) and (max-width: 700px)	@media (min-width: 500px) and (max-width: 700px) { body { background: lightblue; } }	Styles apply strictly between 500px and 700px viewport widths.
Negation Example	@media not (orientation: landscape)	@media not (orientation: landscape) { body { background: pink; } }	Excludes landscape orientation (applies only to portrait mode).
'only' Keyword Example	@media only screen and (max-width: 600px)	@media only screen and (max-width: 600px) { ... }	Ensures older browsers ignore unsupported media queries.
Device Adaptation (Phone)	@media screen and (max-width: 400px)	body { background: orange; color: white; }	Styles for small mobile screens like smartphones.
Tablet View Example	@media screen and (min-width: 401px) and (max-width: 1024px)	body { background: pink; color: blue; }	Styles tailored for tablets or medium resolution devices.

- **Default Media Type Behavior: If you don't specify a media type like screen, the media query applies to all media types (by default = all).**
- **Targeting Screens Specifically:** If you do specify screen, it applies only when the output device is a screen (computer, tablet, mobile, etc.) not printers or other media types.